

MixBytes()

1inch Aqua & Swap-VM Security Audit Report

MARCH 27, 2026

Table of Contents

1. Introduction	3
1.1 Disclaimer	3
1.2 Executive Summary	3
Architecture observations	4
Maker and taker security considerations	4
Protocol and economic observations	5
1.3 Project Overview	6
1.4 Security Assessment Methodology	10
1.5 Risk Classification	12
1.6 Summary of Findings	13
2. Findings Report	15
2.1 Critical	15
C-1 PeggedSwap uses fixed x0/y0 regardless of swap direction, leading to incorrect pricing on reverse swaps	15
2.2 High	16
2.3 Medium	16
M-1 Protocol fee + DynamicBalances causes virtual balance drift from actual position	16
M-2 Aqua protocol fee pull breaks balance check for non-push transfer-in flow	17
M-3 Decay offset grieving via repeated small swaps	18
M-4 XYCConcentrate stores a single liquidity value per orderHash, breaking multi-token strategies	19
M-5 Extruction allows makers to return different swap results in quote vs swap, enabling bait-and-switch	20
M-6 Liquidity distortion when combining XYCConcentrate with Decay	21
2.4 Low	22
L-1 Fee division by zero when feeBps == BPS in exact-out	22

L-2 XYCConcentrate computeDeltas reverts near price boundaries due to exact comparison	23
L-3 Shipped and Docked events can be emitted with empty token arrays	24
L-4 Conditional jumps can cause taker args desynchronization with Extruction calls	25
3. About MixBytes	26

1. Introduction

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

The protocol is a gas-efficient DEX framework built on two core components: Aqua, a shared liquidity layer where Makers deposit assets into strategies, and SwapVM, a bytecode virtual machine for order execution. SwapVM allows Makers to assemble trading strategies – pricing curves, fees, decay, concentration – entirely from calldata-encoded instructions, eliminating the need to deploy new smart contracts. The protocol supports EIP-712 signed orders with standard token transfers as well as Aqua-backed orders where liquidity is managed through the Aqua accounting layer.

The interim audit was carried out over 14 days by a team of 4 auditors.

The following attack vectors were thoroughly examined during the security audit:

Aqua push/pull accounting manipulation. Aqua is a pure accounting layer – it never holds tokens. `push()` transfers tokens from caller to maker and increments the tracked balance; `pull()` transfers from maker to recipient and decrements it. Only the registered app (`msg.sender`) can call `pull()`. `SafeCast.toUint248()` prevents balance overflow on push, and checked subtraction prevents pulling more than available. No path exists to inflate or drain balances beyond what was actually transferred.

Reentrancy into Aqua during swap execution. SwapVM holds a per-orderHash TransientLock for the entire `swap()` call. AquaApp adds per-maker-per-strategy transient locks. `push()` requires the caller to supply real tokens via `safeTransferFrom`, and `pull()` is restricted to `msg.sender == app`. Combined, these prevent any reentrancy path that could manipulate Aqua balances mid-swap or re-enter the same order.

Signature replay and order hash uniqueness. Orders are identified by EIP-712 typed data hash including `maker`, `traits`, and `keccak256(data)`. The `_salt` control instruction allows makers to add arbitrary salt bytes to the program, ensuring unique order hashes for otherwise identical orders. For Aqua mode, `keccak256(abi.encode(order))` is used. The TransientLock per orderHash prevents same-order reentrancy within a transaction.

Same-token swap. We investigated whether `tokenIn == tokenOut` could lead to exploitable behavior. `MakerTraitsLib.validate()` explicitly requires `tokenIn != tokenOut` and reverts with `MakerTraitsTokenInAndTokenOutMustBeDifferent()`. This check runs in both `quote()` and `swap()` after `runLoop()`, preventing any same-token swap from completing regardless of instruction-level behavior.

VM backward jumps and infinite loops via `_jump()`. A maker's program can set `nextPC` to a value less than the current PC, creating backward jumps and potentially infinite loops. This only exhausts gas for the taker executing the order – the transaction reverts when gas runs out. Takers mitigate this by simulating via `quote()` before submitting, which reveals excessive gas consumption. No funds are at risk.

PeggedSwap rounding exploitation and swap-splitting attacks. All four rounding steps in PeggedSwap consistently favor the maker: `u1` rounded down, `solve()` uses `Rounding.Ceil` on the sqrt discriminant, `y1` uses `ceilDiv`, and final `amountOut` uses floor division. Exact-out mirrors this with `amountIn` rounded up. Splitting a single swap into N smaller ones accumulates ~N units of rounding loss for the taker (not the maker), so swap-splitting cannot extract value. The invariant $f(u,v) = \sqrt{u} + \sqrt{v} + A(u+v)$ is strictly increasing in both variables (partial derivatives always positive for positive reserves and $A \geq 0$), guaranteeing the AMM curve cannot be bypassed.

Solidity transient storage clearing helper collision (0.8.28 – 0.8.33 with `--via-ir`). All three repos compile with `via_ir = true` on Solidity 0.8.30, within the affected range. The codebase is **not affected** because `TransientLib` uses inline assembly (`tload/tstore`) directly and never uses Solidity's `delete` on transient state. However, future refactoring that introduces `delete` on `TransientLock` or `uint256` fields would silently break reentrancy guards.

Architecture observations

Instruction ordering affects swap results. The relative placement of Fee, Decay, Concentrate, and Swap instructions materially changes the computed amounts – e.g. Fee before Swap taxes the raw input, Fee after Swap taxes the output. Makers and order auditors should verify that instruction ordering matches the intended economic behavior.

Opcodes and data are not separated. The VM bytecode interleaves opcodes and inline arguments with no structural boundary. The `_jump` instruction can set PC into the middle of another instruction's arguments, causing those bytes to execute as opcodes. This enables order obfuscation: an order appears benign, but a Jump into argument data activates hidden logic. Order auditors and whitelist systems must trace all reachable code paths, including Jump targets within argument regions.

Reusing programs across opcode sets is dangerous. A SwapVM program is interpreted against a specific opcode set. The three existing sets – `Opcodes`, `AquaOpcodes`, and `LimitOpcodes` – share Control instructions at indices 11–17 but diverge completely afterward (e.g. index 18 is `_staticBalancesXD` in `Opcodes` but `_xycSwapXD` in `AquaOpcodes`; index 23 is `_xycSwapXD` / `_decayXD` / `_requireMinRate1D` respectively). A program designed for one opcode set will silently execute wrong instructions on another, making strategy reuse across sets unsafe.

Maker and taker security considerations

Cross-swap phishing via non-isolated token pairs. In Aqua, when a maker creates multiple strategies (e.g. USDC/USDT and a low-liquidity token) without distinct salts, unintended trading pairs form between tokens across strategies. A phishing frontend can trick a maker into signing a single `ship` for a worthless token, which then enables draining the maker's legitimate pair. Salt should be made mandatory or pairs should be architecturally isolated.

Extraction is a wildcard instruction. `_extraction` delegates execution to an arbitrary external contract specified by the maker, making it the most powerful and least auditable instruction in the VM. The external contract can contain any logic – price manipulation, gas-expensive computations that make order execution unprofitable for takers, or state-dependent behavior invisible to static analysis. For example, since `isStaticContext` is passed to the external contract, the maker can distinguish `quote()` from `swap()` and return attractive prices during quoting while delivering worse rates during execution.

Threshold defines maker's maximum deduction from taker. The taker-side threshold sets the worst acceptable rate. If misconfigured (e.g. set to zero or an overly permissive value), the maker's program can extract up to that amount from the taker. Takers and integrating frontends should pay particular attention to threshold configuration, as it is the last line of defense against unfavorable execution.

Protocol and economic observations

No on-chain verification of maker's declared liquidity (Aqua). In Aqua mode, a maker self-reports its available liquidity in the program with no on-chain enforcement mechanism. There is nothing preventing a maker with minimal funds from advertising arbitrarily large liquidity. Aggregators and resolvers should implement off-chain liquidity verification before routing volume to a maker.

Fee rounds down (taker-friendly). `_feeAmountIn` computes fee as `amountIn * feeBps / BPS` using floor division, which consistently rounds in the taker's favor. For small `amountIn` values where `amountIn * feeBps < BPS`, the fee truncates to zero – the taker pays nothing. While not economically exploitable on mainnet (gas cost far exceeds the rounding benefit), the relative fee loss for the maker grows as `amountIn` decreases: e.g. with `feeBps = 1e6` (0.1%), any `amountIn < 1000` yields zero fee. On cheap L2s with sub-cent gas, this becomes more relevant for low-decimal tokens.

Simulator only works via self-delegatecall. The `Simulator.simulate()` function reads storage slots that are only meaningful when called via `delegatecall` from the inheriting contract itself (e.g. `AquaSwapVMRouter`). If called from a different contract, storage layout will differ and simulation results will be incorrect. Consider adding an `address(this)` check to prevent misuse.

Unwrap does not verify the token is WETH. The `shouldUnwrapWeth` trait flag (on both maker and taker side) triggers `IWETH(token).safeWithdrawTo()` in `_transferFrom()` without checking that `token` matches the canonical WETH address passed to the constructor. If the flag is set for a non-WETH token, the call will revert (no `withdraw()` function). This is self-harming only – the flag is controlled by the party setting it – but a check against the known WETH address would prevent wasted gas on misconfigured orders.

Aqua ship() allows adding extra tokens to an existing strategy. `Aqua.ship()` can be called multiple times with the same `strategyHash` (derived from the same `strategy` bytes) but with different token arrays. The described scenario is addressed in `README.md` under "Strategy Hash Uniqueness and Token Safety" section. The vulnerability exists only when makers violate documented best practices: 1. Non-Aqua mode – Programs without `_staticBalancesXD` or `_dynamicBalancesXD` instructions; 2. Aqua mode – Programs without token-specific `_salt` or validation in custom instructions.

A mechanism to recover tokens stuck in Aqua and SwapVM. During the audit, there was identified a finding related to Aqua contract and a missing rescue or sweep function for tokens which got stuck on the contract. There was a comment from the developers of the protocol that Aqua is an accounting layer, not a custody protocol. Tokens always remain in the maker's wallet. `ship()` registers balances but does not transfer tokens. `pull()` executes `transferFrom(maker, ...)` from the maker's address. Accidental sends – is responsibility of sender. Though, there was a `Rescuable` functionality added to the Aqua and SwapVM contract, which was reviewed during the re-audit and confirmed to be safe as the `rescueFunds` function defined in the solidity-utils repo is protected with the `onlyOwner` modifier and there were no potential reentrancy attack vectors identified.

The audit focused on the Aqua liquidity layer, the core SwapVM engine, and the `AquaOpcodes` instruction set. The following SwapVM components were **not** in scope: instructions `BaseFeeAdjuster`, `Debug`, `DutchAuction`, `FeeExperimental`, `Invalidators`, `LimitSwap`, `MinRate`, `OraclePriceAdjuster`, `TWAPSwap`, and `XYCConcentrateExperimental`; opcode sets `Opcodes` and `LimitOpcodes`; routers `SwapVMRouter` and `LimitSwapVMRouter`. These components may carry additional risks not covered by this audit.

The SwapVM and Aqua codebase demonstrates a well-engineered architecture with a compact bytecode VM, clean instruction separation, and correct use of transient storage for reentrancy protection. However, the audit identified critical issue in PeggedSwap's direction-dependent math (asymmetric invariant between forward and reverse swaps), as well as problematic interactions between Decay and XYCConcentrate instructions when combined in multi-token strategies. These and other findings are detailed in the report below.

1.3 Project Overview

Summary

Title	Description
Client	1inch
Category	DEX Aggregator
Project	Aqua & Swap-VM
Type	Solidity
Platform	EVM
Timeline	03.02.2026 – 27.03.2026

Scope of Audit

File	Link
<code>src/AquaRouter.sol</code>	src/AquaRouter.sol
<code>src/AquaApp.sol</code>	src/AquaApp.sol
<code>src/Aqua.sol</code>	src/Aqua.sol
<code>src/libs/Balance.sol</code>	src/libs/Balance.sol
<code>src/SwapVM.sol</code>	src/SwapVM.sol
<code>src/routers/AquaSwapVMRouter.sol</code>	src/routers/AquaSwapVMRouter.sol
<code>src/opcodes/AquaOpcodes.sol</code>	src/opcodes/AquaOpcodes.sol
<code>src/instructions/Balances.sol</code>	src/instructions/Balances.sol
<code>src/instructions/PeggedSwap.sol</code>	src/instructions/PeggedSwap.sol
<code>src/instructions/XYCSwap.sol</code>	src/instructions/XYCSwap.sol
<code>src/instructions/Fee.sol</code>	src/instructions/Fee.sol
<code>src/instructions/XYCConcentrate.sol</code>	src/instructions/XYCConcentrate.sol
<code>src/instructions/Extruction.sol</code>	src/instructions/Extruction.sol
<code>src/instructions/Controls.sol</code>	src/instructions/Controls.sol
<code>src/instructions/Decay.sol</code>	src/instructions/Decay.sol
<code>src/libs/PeggedSwapMath.sol</code>	src/libs/PeggedSwapMath.sol
<code>src/libs/VM.sol</code>	src/libs/VM.sol
<code>src/libs/MakerTraits.sol</code>	src/libs/MakerTraits.sol
<code>src/libs/TakerTraits.sol</code>	src/libs/TakerTraits.sol
<code>contracts/libraries/CalldataPtr.sol</code>	contracts/libraries/CalldataPtr.sol
<code>contracts/libraries/Transient.sol</code>	contracts/libraries/Transient.sol
<code>contracts/libraries/Calldata.sol</code>	contracts/libraries/Calldata.sol

File	Link
<code>contracts/libraries/TransientLock.sol</code>	contracts/libraries/TransientLock.sol
<code>contracts/mixins/Multicall.sol</code>	contracts/mixins/Multicall.sol
<code>contracts/mixins/Simulator.sol</code>	contracts/mixins/Simulator.sol
<code>contracts/mixins/Rescuable.sol</code>	contracts/mixins/Rescuable.sol

Versions Log

Date	Commit Hash	Note
03.02.2026	89cedfa608aa9fdc60d8cd936b76a540e4b9346f	Initial Commit (Aqua)
03.02.2026	e2de56b7b7c759b04060c3b4535f6e489c09a536	Initial Commit (Swap VM)
03.02.2026	38e5f8de705a8821237520ae06f3e9b0d40daf01	Initial Commit (Solidity Utils)
12.03.2026	af53fc31b636c683a6b72cf4755f5aad089c12e8	Commit for Re-audit (Aqua)
12.03.2026	f6631a0b880d302950336babd3d227c93dee1920	Commit for Re-audit (Swap VM)
12.03.2026	29043f22422fde454951e9733129cce5d67e6a39	Commit for Re-audit (Solidity Utils)
18.03.2026	81c26e4619ce21556ab02b3284ee2685de21fb18	Commit with Updates (Aqua)
18.03.2026	00ce47af6e456a91f9236bf7e12ac4216195e144	Commit with Updates (Swap VM)

Date	Commit Hash	Note
18.03.2026	81ad207e52c1d43b8fe5ca8bdb876d108746f867	Commit with Updates (Solidity Utils)
27.03.2026	b2daef83656b1ccbfcc1c83d3df869c0ebb60badc	Commit with Updates (Swap VM)

Mainnet Deployments

Deployment verification will occur after the smart contracts are deployed on the mainnet.

1.4 Security Assessment Methodology

Project Flow

Stage	Scope of Work
Interim audit	Project Architecture Review: • Review project documentation • Conduct a general code review • Perform reverse engineering to analyze the project's architecture based solely on the source code • Develop an independent perspective on the project's architecture • Identify any logical flaws in the design OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROJECT AND IDENTIFY POTENTIAL SECURITY RISKS.
	Code Review with a Hacker Mindset: • Each team member independently conducts a manual code review, focusing on identifying unique vulnerabilities. • Perform collaborative audits (pair auditing) of the most complex code sections, supervised by the Team Lead. • Develop Proof-of-Concepts (PoCs) and conduct fuzzing tests using tools like Foundry, Hardhat, and BOA to uncover intricate logical flaws. • Review test cases and in-code comments to identify potential weaknesses. OBJECTIVE: IDENTIFY AND ELIMINATE THE MAJORITY OF VULNERABILITIES, INCLUDING THOSE UNIQUE TO THE INDUSTRY.
	Code Review with a Nerd Mindset: • Conduct a manual code review using an internally maintained checklist, regularly updated with insights from past hacks, research, and client audits. • Utilize static analysis tools (e.g., Slither, Mythril) and vulnerability databases (e.g., Solodit) to uncover potential undetected attack vectors. OBJECTIVE: ENSURE COMPREHENSIVE COVERAGE OF ALL KNOWN ATTACK VECTORS DURING THE REVIEW PROCESS.

Stage	Scope of Work
	Consolidation of Auditors' Reports: • Cross-check findings among auditors • Discuss identified issues • Issue an interim audit report for client review OBJECTIVE: COMBINE INTERIM REPORTS FROM ALL AUDITORS INTO A SINGLE COMPREHENSIVE DOCUMENT.
Re-audit	Bug Fixing & Re-Audit: • The client addresses the identified issues and provides feedback • Auditors verify the fixes and update their statuses with supporting evidence • A re-audit report is generated and shared with the client OBJECTIVE: VALIDATE THE FIXES AND REASSESS THE CODE TO ENSURE ALL VULNERABILITIES ARE RESOLVED AND NO NEW VULNERABILITIES ARE ADDED.
Final audit	Final Code Verification & Public Audit Report: • Verify the final code version against recommendations and their statuses • Check deployed contracts for correct initialization parameters • Confirm that the deployed code matches the audited version • Issue a public audit report, published on our official GitHub repository • Announce the successful audit on our official X account OBJECTIVE: PERFORM A FINAL REVIEW AND ISSUE A PUBLIC REPORT DOCUMENTING THE AUDIT.

1.5 Risk Classification

Severity Level Matrix

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** – Theft from 0.5% OR partial/full blocking of funds (>0.5%) on the contract without the possibility of withdrawal OR loss of user funds (>1%) who interacted with the protocol.
- **Medium** – Contract lock that can only be fixed through a contract upgrade OR one-time theft of rewards or an amount up to 0.5% of the protocol's TVL OR funds lock with the possibility of withdrawal by an admin.
- **Low** – One-time contract lock that can be fixed by the administrator without a contract upgrade.

Likelihood

- **High** – The event has a 50-60% probability of occurring within a year and can be triggered by any actor (e.g., due to a likely market condition that the actor cannot influence).
- **Medium** – An unlikely event (10-20% probability of occurring) that can be triggered by a trusted actor.
- **Low** – A highly unlikely event that can only be triggered by the owner.

Action Required

- **Critical** – Must be fixed as soon as possible.
- **High** – Strongly advised to be fixed to minimize potential risks.
- **Medium** – Recommended to be fixed to enhance security and stability.
- **Low** – Recommended to be fixed to improve overall robustness and effectiveness.

Finding Status

- **Fixed** – The recommended fixes have been implemented in the project code and no longer impact its security.
- **Partially Fixed** – The recommended fixes have been partially implemented, reducing the impact of the finding, but it has not been fully resolved.
- **Acknowledged** – The recommended fixes have not yet been implemented, and the finding remains unresolved or does not require code changes.

1.6 Summary of Findings

Findings Count

Severity	Count
Critical	1
High	0
Medium	6
Low	4

Findings Statuses

ID	Finding	Severity	Status
C-1	PeggedSwap uses fixed x0/y0 regardless of swap direction, leading to incorrect pricing on reverse swaps	Critical	Fixed
M-1	Protocol fee + DynamicBalances causes virtual balance drift from actual position	Medium	Fixed
M-2	Aqua protocol fee pull breaks balance check for non-push transfer-in flow	Medium	Fixed
M-3	Decay offset griefing via repeated small swaps	Medium	Acknowledged
M-4	XYCConcentrate stores a single liquidity value per orderHash, breaking multi-token strategies	Medium	Fixed
M-5	Extraction allows makers to return different swap results in quote vs swap, enabling bait-and-switch	Medium	Fixed
M-6	Liquidity distortion when combining XYCConcentrate with Decay	Medium	Fixed
L-1	Fee division by zero when feeBps == BPS in exact-out	Low	Acknowledged

L-2	XYCConcentrate computeDeltas reverts near price boundaries due to exact comparison	Low	Fixed
L-3	Shipped and Docked events can be emitted with empty token arrays	Low	Acknowledged
L-4	Conditional jumps can cause taker args desynchronization with Extruction calls	Low	Acknowledged

2. Findings Report

2.1 Critical

C-1	PeggedSwap uses fixed x0/y0 regardless of swap direction, leading to incorrect pricing on reverse swaps		
Severity	Critical	Status	Fixed in f6631a0b

Description

In `PeggedSwap._peggedSwapGrowPriceRange2D()`, the config parameters `x0` and `y0` are hardcoded in the program bytecode and represent the initial reserves of the "first" and "second" token respectively. Rate multipliers (`rateLt/rateGt`) are correctly swapped based on token address comparison via `getRates()`, but `config.x0` and `config.y0` are not.

When a forward swap (A→B) occurs, `balanceIn` corresponds to token A and is normalized by `config.x0` (which was set as token A's initial reserve). This is correct. But on a reverse swap (B→A), `balanceIn` is now token B's balance, yet it's still normalized by `config.x0` – which is token A's initial reserve. The invariant calculation `u = balanceIn * rateIn * ONE / config.x0` uses the wrong normalization factor, producing an incorrect curve.

With different initial balances or different rate multipliers between the two tokens, this mismatch causes the invariant to shift between swap directions. An attacker can exploit this by repeatedly swapping back and forth, extracting value from the curve's inconsistency. At strong imbalances (e.g., `x0 = 2 * y0`), draining the maker's position requires as few as ~12 swap iterations.

Recommendation

Make `x0/y0` direction-aware. Either swap them based on `tokenIn < tokenOut` (same logic as rates), or store the initial reserves keyed by token address rather than by fixed position. Alternatively, enforce a convention that `x0` always corresponds to the token with the lower address and document this requirement clearly.

Client's Commentary:

Fixed in [PR-68](#)

2.2 High

Not Found

2.3 Medium

M-1	Protocol fee + DynamicBalances causes virtual balance drift from actual position		
Severity	Medium	Status	Fixed in f6631a0b

Description

When protocol fee instructions (`_protocolFeeAmountInXD`, `_aquaProtocolFeeAmountInXD`, and their out variants) are combined with `_dynamicBalancesXD`, the DynamicBalances storage accumulates a growing discrepancy with the maker's actual token position.

Fee instructions save the original `amountIn`, reduce it for the inner swap, run the swap, then restore the original value. After return, DynamicBalances records the restored (fee-inclusive) amount: `balances[tokenIn] += swapAmountIn`. But the protocol fee transfer already pulled `feeAmountIn` from the maker. So DynamicBalances records the full amount while the maker only retains `amountIn - feeAmountIn`.

After N swaps, DynamicBalances is inflated by the cumulative protocol fees. The AMM then quotes prices as if the pool is deeper than it actually is. Reverse-direction swaps eventually fail because the maker lacks tokens to cover the inflated balance, causing a silent DoS.

This does NOT affect flat fee instructions (`_flatFeeAmountInXD`, `_flatFeeAmountOutXD`) since those don't transfer tokens out.

Recommendation

After the fee instruction restores the original amount, subtract the fee amount from the value that DynamicBalances will record. Alternatively, have fee instructions report the actual net amount rather than restoring the pre-fee value.

Client's Commentary:

Client: Correct order and combinations of instructions described in [readme.md](#). Also added additional register for correct accounting with Aqua: 9b9d821eb2f4ed8e2ffa00a4ccb161ca3cef7558

MixBytes: We also recommend fixing the order of operations in tests. E.g. in `ConcentrateXYCDelayFeesInvariants.test_Order4_GrowLiquidity2D()` `_protocolFeeAmountOutXD` is used after `_dynamicBalancesXD`.

Client: Fixed in [PR-75](#)

M-2	Aqua protocol fee pull breaks balance check for non-push transfer-in flow		
Severity	Medium	Status	Fixed in f6631a0b

Description

In `SwapVM.swap()`, when the order uses Aqua mode and `useTransferFromAndAquaPush` is false, the taker is expected to push tokens to the maker's Aqua balance via a callback. After the swap VM executes, `_transferIn()` checks: `balanceIn >= originalAquaBalanceIn + ctx.swap.amountIn`. However, if the VM program includes `_aquaProtocolFeeAmountInXD`, this instruction calls `AQUA.pull()` during execution, reducing the maker's Aqua balance for `tokenIn`. The `originalAquaBalanceIn` was snapshotted before the VM ran. After the fee pull, even if the taker pushed enough tokens, the current `balanceIn` is reduced by the fee amount and the check fails.

This makes the combination of `useTransferFromAndAquaPush = false + hasPreTransferInCallback = true + Aqua protocol fee on tokenIn` effectively a permanent DoS for that configuration.

Recommendation

Snapshot `originalAquaBalanceIn` after the VM execution completes (i.e., after all fee pulls have happened), or adjust the balance check to account for protocol fees pulled during execution.

Client's Commentary:

Correct order and combinations of instructions described in [readme.md](#). Also added additional register for correct accounting with Aqua: 9b9d821eb2f4ed8e2ffa00a4ccb161ca3cef7558

Fixed in [PR-75](#)

M-3	Decay offset griefing via repeated small swaps		
Severity	Medium	Status	Acknowledged

Description

In `DecayingOffsetLib.addOffset()`, the existing decayed offset is summed with the new swap's offset, and `block.timestamp` is reset. This means each new swap extends the effective decay tail: the old offset that was partially decayed gets "refreshed" back to full decay period. An attacker can exploit this on cheap L2s by performing frequent small swaps in one direction. Each swap adds a small offset but resets the timer, preventing previous offsets from decaying. Over time, the accumulated offset grows large enough to significantly shift the effective balance, blocking the strategy for the reverse direction (since `balanceOut -= offset` will revert on underflow) or severely mispricing it. The cost of this attack depends on the chain's gas costs and the swap fees. On L2s with sub-cent transactions, the attacker can sustain this griefing cheaply.

Recommendation

Consider capping the maximum accumulated offset (e.g., as a fraction of the pool balance), or change the decay accumulation so that adding a new offset doesn't fully reset the timer for the pre-existing portion. For example, use a weighted-average timestamp instead of always resetting to `block.timestamp`.

Client's Commentary:

ACKNOWLEDGED - Dust swaps can extend decay period, but offset magnitude decreases with each refresh. Gas costs make attack unprofitable; impact naturally diminishes over time.

M-4	XYCConcentrate stores a single liquidity value per orderHash, breaking multi-token strategies		
Severity	Medium	Status	Fixed in f6631a0b

Description

`XYCConcentrate.liquidity` is a `mapping(bytes32 orderHash => uint256)` – a single liquidity value per order. When a maker has more than 2 tokens in a strategy and uses `_xycConcentrateGrowLiquidityXD`, each swap updates this shared liquidity value based on the specific token pair involved.

For example, with tokens A, B, C: a swap A→B computes `newInv = (balanceA + amountIn) * (balanceB - amountOut)` and stores `sqrt(newInv)` in `liquidity[orderHash]`. A subsequent swap B→C reads this A,B-derived liquidity and uses it in `concentratedBalance()` to scale the B,C deltas, which is mathematically incorrect – the liquidity concentration for B,C has nothing to do with the A,B invariant.

Furthermore, it is not entirely clear how concentrated liquidity is supposed to be used for more than two tokens.

Recommendation

We recommend removing `_xycConcentrateGrowLiquidityXD` function and documenting that concentrated liquidity can only be used for dual-token swaps.

Client's Commentary:

Fixed in [PR-82](#)

M-5	Extruction allows makers to return different swap results in quote vs swap, enabling bait-and-switch		
Severity	Medium	Status	Fixed in f6631a0b

Description

The `Extruction` instruction passes `isStaticContext` to the external target contract. A malicious maker can deploy an extruction target that checks this flag and returns favorable swap amounts during `quote()` (where `isStaticContext = true`) but unfavorable amounts during `swap()` (where `isStaticContext = false`).

Aggregators and takers rely on `quote()` to estimate swap outcomes before executing. If the quoted price looks competitive, the taker submits the transaction. During actual execution, the extruction returns a worse price, and the taker receives less than expected. The taker's threshold setting provides the only protection – if it's unset or loose, the taker loses funds up to the threshold tolerance.

Note: Extruction is currently not registered in `AquaOpcodes._opcodes()`, so this is not exploitable through the current `AquaSwapVMRouter`. It becomes relevant if Extruction is added to a production opcode set.

Recommendation

Remove the `isStaticContext` parameter from the `IExtruction` interface, or enforce that extruction targets are whitelisted/audited. Alternatively, document clearly that takers MUST set tight threshold amounts when interacting with orders that use Extruction.

Client's Commentary:

Added docs about this behavior: 9b9d821eb2f4ed8e2ffa00a4ccb161ca3cef7558

Fixed in [PR-84](#)

M-6	Liquidity distortion when combining XYCConcentrate with Decay		
Severity	Medium	Status	Fixed in f6631a0b

Description

This issue has been identified within the `_decayXD` function of the `Decay` contract. When `XYCConcentrate` is used together with `Decay`, the stored `liquidity` value may be recalculated using distorted `balanceIn` and `balanceOut` values produced by the decay adjustment. As a result, subsequent swaps (even after the decay period has elapsed) may use an increasingly incorrect liquidity value, and the distortion can accumulate over time. This can impact the amounts received by takers and may lead to losses for makers. The issue is classified as Medium severity because it can materially affect pricing and swap outcomes over time and may cause economic loss, but it requires a specific instruction composition (`XYCConcentrate` combined with `Decay`) and manifests across subsequent swaps rather than as an immediate, universal failure.

Recommendation

We recommend allowing the decay logic only within the concentrate opcode (i.e., have concentrate call decay internally), and ensure that `Decay._decayXD()` restores balances to their non-distorted values before returning so that `liquidity` is computed from the correct (undistorted) balances.

Client's Commentary:

Client: Correct order and combinations of instructions described in [readme.md](#). Stateless XYCConcentrate version: 9b9d821eb2f4ed8e2ffa00a4ccb161ca3cef7558

MixBytes: We also recommend fixing the order of operations in tests. E.g. in `ConcentrateXYCDecayInvariants.test_ConcentrateGrowLiquidity2D_Decay()` `_decayXD` is used after `_xycConcentrateGrowLiquidity2D`.

Client: Fixed in [PR-75](#) and [PR-82](#)

2.4 Low

L-1	Fee division by zero when feeBps == BPS in exact-out		
Severity	Low	Status	Acknowledged

Description

In `Fee._feeAmountIn()` (line 196), the exact-out formula is `feeAmountIn = ctx.swap.amountIn * feeBps / (BPS - feeBps)`. When `feeBps == BPS` (100%), the denominator is zero and the transaction reverts with a generic panic. Same issue exists in `FeeExperimental._feeAmountOut()` line 138. The builder allows `feeBps <= BPS`, so 100% passes validation.

Exact-in with 100% fee works fine (gives 0 output), but exact-out silently reverts. Additionally, `parseFlatFee()` does no range validation, so a maker could embed `feeBps > BPS` in order data, causing underflows in exact-in or near-zero fees in exact-out.

Recommendation

Add an explicit check for `feeBps == BPS` in the exact-out path (revert with a descriptive error or handle it as a special case). Validate `feeBps <= BPS` in `parseFlatFee()` the same way `buildFlatFee()` does.

Client's Commentary:

We would not want to introduce a check condition that would always pass except for obviously erroneous orders, while adding gas costs to all orders. Takers must validate strategies before use.

L-2	XYCConcentrate computeDeltas reverts near price boundaries due to exact comparison		
Severity	Low	Status	Fixed in f6631a0b

Description

In `XYCConcentrateArgsBuilder.computeDeltas()`, when `price == priceMin` or `price == priceMax`, the delta is set to 0 via exact equality checks. However, due to integer square root rounding, `Math.sqrt(price * ONE / priceMin) * SQRT_ONE` can equal `ONE` (making the denominator `sqrtPriceMin - ONE = 0` and causing division by zero) for prices slightly above `priceMin` – within ~2 wei for 18-decimal tokens. This causes unexpected reverts on the frontend/builder side when prices are near the concentration range boundaries.

Recommendation

Add a small tolerance band around the boundary prices, or handle the case where the sqrt result rounds to exactly `ONE` by treating it as the boundary case (`delta = 0`).

Client's Commentary:

computeDeltas() is outdated. Fixed: 9b9d821eb2f4ed8e2ffa00a4ccb161ca3cef7558

Fixed in [PR-82](#)

L-3	Shipped and Docked events can be emitted with empty token arrays		
Severity	Low	Status	Acknowledged

Description

`Aqua.ship()` does not check that `tokens.length > 0` before emitting the `Shipped` event. Similarly, `dock()` does not verify a non-empty array. While a zero-length ship creates an unusable strategy (since `safeBalances` rejects `tokensCount == 0`), the emitted events can confuse indexers, subgraphs, and frontend applications that track strategy lifecycle.

Recommendation

Add `require(tokens.length > 0)` at the start of both `ship()` and `dock()`.

Client's Commentary:

ACKNOWLEDGED - Indexers, integrators, aggregators and applications should check event parameters.

L-4	Conditional jumps can cause taker args desynchronization with Extruction calls		
Severity	Low	Status	Acknowledged

Description

Extruction consumes taker-provided arguments via `ctx.tryChopTakerArgs(choppedLength)`, which linearly advances through a flat byte buffer. When the maker's program uses conditional jumps (`_jumpIfTokenIn`, `_jumpIfTokenOut`) that can skip over an Extruction call, the taker's argument buffer gets out of sync – data intended for extruction N+1 is consumed by extruction N, or vice versa.

The taker can account for this if they know the exact execution path (which depends on `tokenIn/tokenOut`), but the design is fragile and error-prone. Simulation via `quote()` would catch the mismatch, but a subtle off-by-one in arg construction could lead to a successful but incorrect execution.

Note: Extruction is not currently registered in [AquaOpcodes](#), so this only becomes relevant if it's added to a production opcode set.

Re-audit update: Extruction is now registered in [AquaOpcodes](#) (commit [f6631a0b880d302950336babd3d227c93dee1920](#)).

Recommendation

Consider labeling taker args with the target instruction index or PC offset, so that each Extruction consumes only the data explicitly intended for it, regardless of which branches were taken.

Client's Commentary:

Client: Extruction added to AquaOpcodes: [9b9d821eb2f4ed8e2ffa00a4ccb161ca3cef7558](#). Taker should verify every strategy.

MixBytes: While takers can simulate the execution path via `quote()` beforehand, the issue is amplified when multiple Extruction calls appear in different conditional branches: taker arguments are consumed linearly from a flat buffer, so the taker cannot deterministically assign argument segments to specific Extruction calls without knowing which branch will execute. Furthermore, Extruction grants full control over `ctx.swap` (including `amountNetPulled`) and `nextPC`, allowing arbitrary modification of swap amounts and control flow.

3. About MixBytes

MixBytes is a leading provider of smart contract auditing and blockchain security research, helping Web3 projects enhance their security and reliability.

Since its inception, MixBytes has been dedicated to safeguarding innovation in **DeFi** through rigorous audits and cutting-edge technical research.

Our team brings together experienced engineers, security auditors, and blockchain researchers with deep expertise in smart contract security and protocol design.

With years of proven experience in Web3, MixBytes combines in-depth technical excellence with a proactive, security-first approach.

Why MixBytes

- **Proven Track Record:** Trusted by leading blockchain protocols such as **Lido**, **Aave**, **Curve**, **1inch**, **Fluid**, **Gearbox**, **Resolv**, and others. MixBytes has successfully secured billions of dollars in digital assets.
- **Technical Expertise:** Our auditors and researchers hold advanced degrees in cryptography, cybersecurity, and distributed systems, backed by years of hands-on experience.
- **Innovative Research:** MixBytes actively contributes to blockchain security research and open-source tools, sharing insights with the global community.

Our Services

- **Smart Contract Audits:** Comprehensive security assessments of DeFi protocols to detect and mitigate vulnerabilities before deployment.
- **Blockchain Research:** In-depth technical studies, economic and protocol-level modeling for Web3 projects.
- **Custom Security Solutions:** Tailored frameworks and advisory for complex decentralized systems and blockchain ecosystems.

Contact Information



<https://mixbytes.io/>



https://github.com/mixbytes/audits_public



<https://x.com/MixBytes>



hello@mixbytes.io